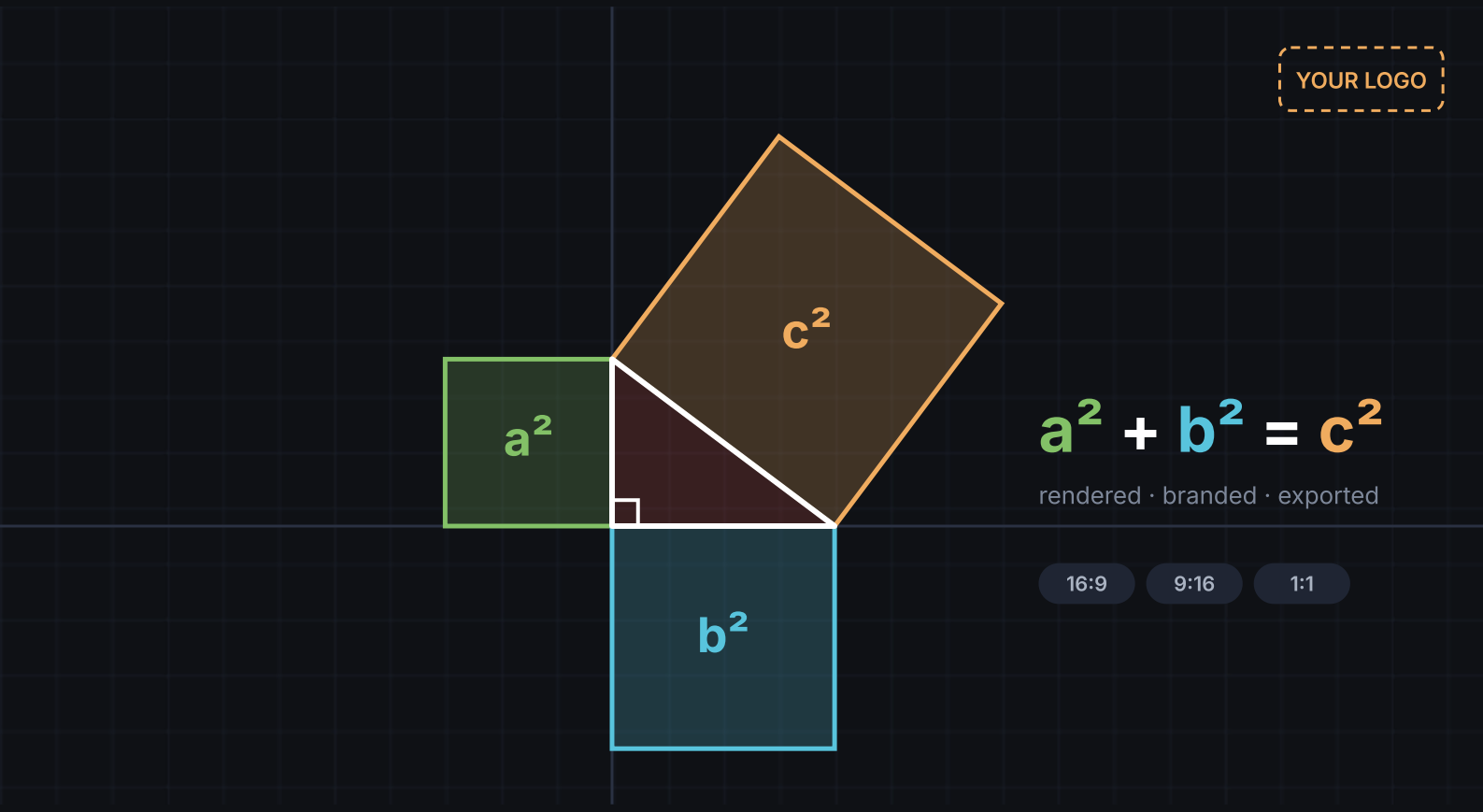
PRODUCT OVERVIEW · V0.1 · OCTOBER 2026

Pi & Pixel

A code-first studio for 3Blue1Brown-style explainer videos. Powered by Manim. Controlled from the browser. Rendered on your own machine or on Google Colab.



1 project → many videos

Every format, brand and language from one source

Zero render servers

Runs on a laptop, in Docker or on Google Colab

Up to 30 min

Validated 1080p MP4 per exported video

Prepared by Harsh Jha · Content engine for ReelMVP

Status: design complete · build starts at stage 🚲 Cycle

01 · AT A GLANCE

Explainer videos as code

Pi & Pixel wraps the Manim animation engine in a browser studio. You write (or later, generate) a scene, preview it, apply a brand, and export every format you need from one project.

The problem

- 3Blue1Brown-style animation needs Python, Manim, FFmpeg and LaTeX set up correctly.
- Rendering is heavy on RAM and CPU, so weak laptops struggle.
- Every format (YouTube, Reels) and every client brand means manual re-work.
- Hosting render servers before the product is validated is expensive.

The solution

- A studio in the browser: editor, live preview, logs, brand kits, export gallery.
- Rendering runs on the user's compute: their PC, Docker or Google Colab.
- One project produces many videos × formats × brands × languages.
- The same render core later runs on our servers inside ReelMVP.

Five ideas that define the product

🖥️

Bring your own compute

The browser only controls. All heavy work happens where the Agent runs: laptop, Docker or Colab.

π

Manim stays Manim

Scenes are plain Manim Python. No hidden engine, no lock-in. They also run with the normal `manim` command.

🎨

Brand kits

Logo position, size, opacity and timing, plus colours and fonts. Moving a logo never re-renders the animation.

🧩

Many videos, many variants

Compositions × formats × brands × parameters. Shared scenes are rendered once and reused.

🚚

ReelMVP-ready

The pipeline is a headless library plus a CLI. ReelMVP's servers will run the same code.

🤖

AI when useful

Later: Claude generates and fixes scenes. Its code is always shown as a diff and never auto-runs.

The workflow

✍️ Write · 👀 Preview · 🐞 Debug · 🎨 Brand · 🎬 Compose · 🔳 Variants · ✅ Validate · 📦 Export

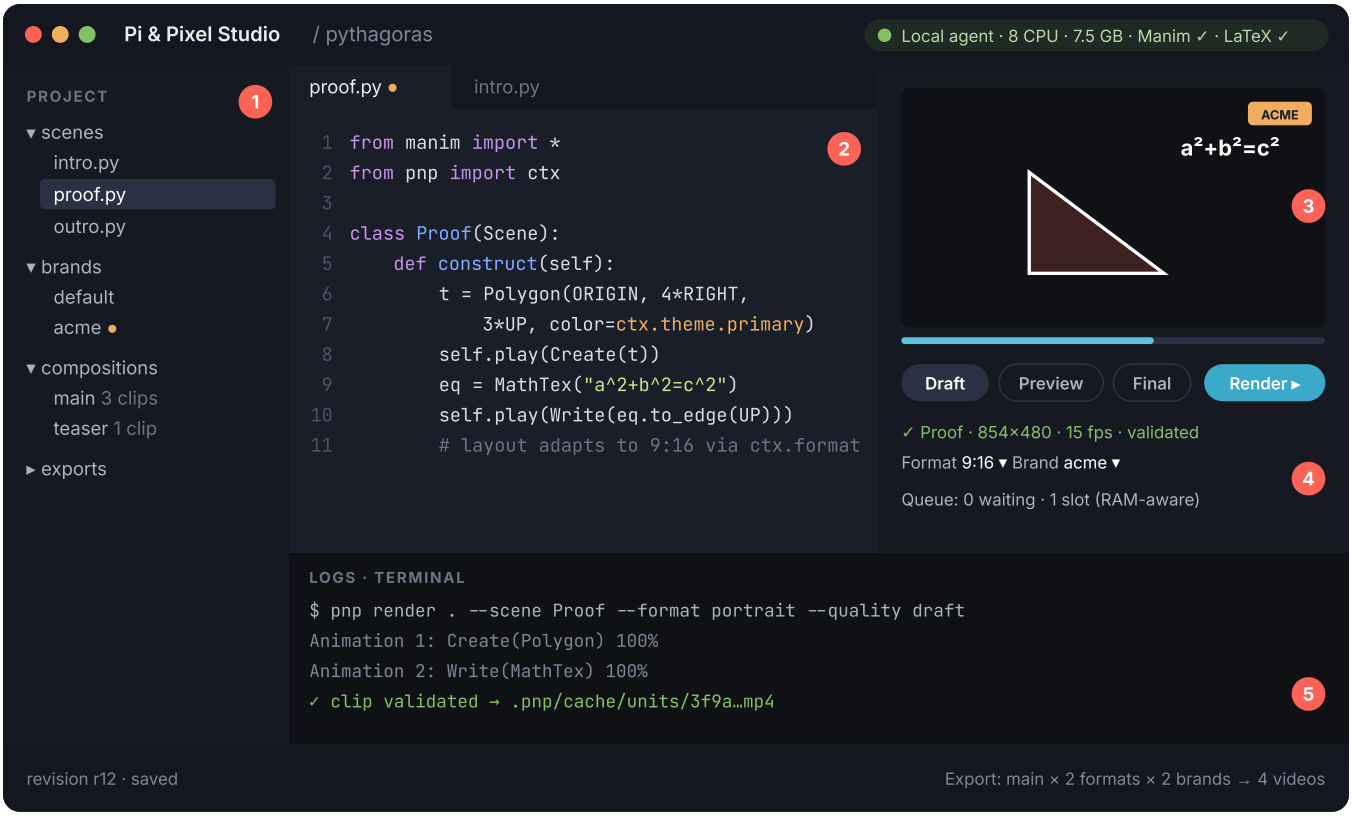
Who it is for

USER	WHAT THEY GET
Technical creators and educators	Full control over Manim code, with fast previews and readable errors
ReelMVP content team	The same lesson as a 16:9 video, a 9:16 reel and a teaser, in any brand, from one project
Users with low-spec laptops	The same studio, rendering on free or paid Google Colab
AI-assisted creators (later)	Describe an animation in words, review the generated code, render it

02 · WHAT YOU SEE

The Studio

A three-panel workspace built for iteration: code on the left, result on the right, and the machine doing the work always visible.



Illustrative mockup of the target Studio. Screens will change during the build.

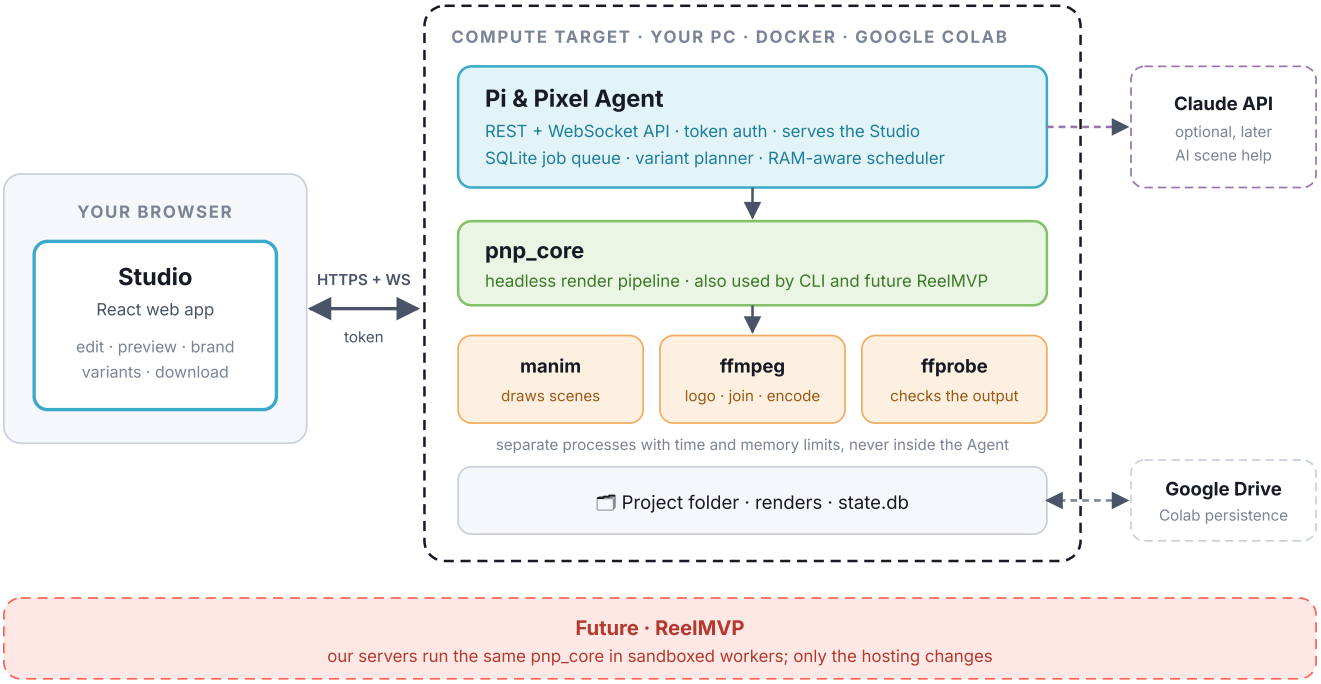
#	AREA	WHAT IT DOES
1	Project explorer	Scene files, brand kits, compositions (videos) and exports in one tree
2	Code editor	Monaco (the VS Code editor): syntax highlighting, tabs, search, errors linked to the line
3	Preview	Plays the latest render for the chosen format and brand; the logo overlay updates live
4	Render controls	Draft / Preview / Final quality, format and brand pickers, a RAM-aware queue
5	Logs and terminal	Live Manim output, validation results, and a terminal for power users (local mode)

The status badge at the top always shows which machine is doing the work and what it can do (CPU, RAM, Manim and LaTeX availability).

03 · ARCHITECTURE

The browser controls. Your machine renders.

Two parts: the **Studio** (a web page) and the **Agent** (a Python program on the compute target). Video data never travels through our servers.



Where it can run

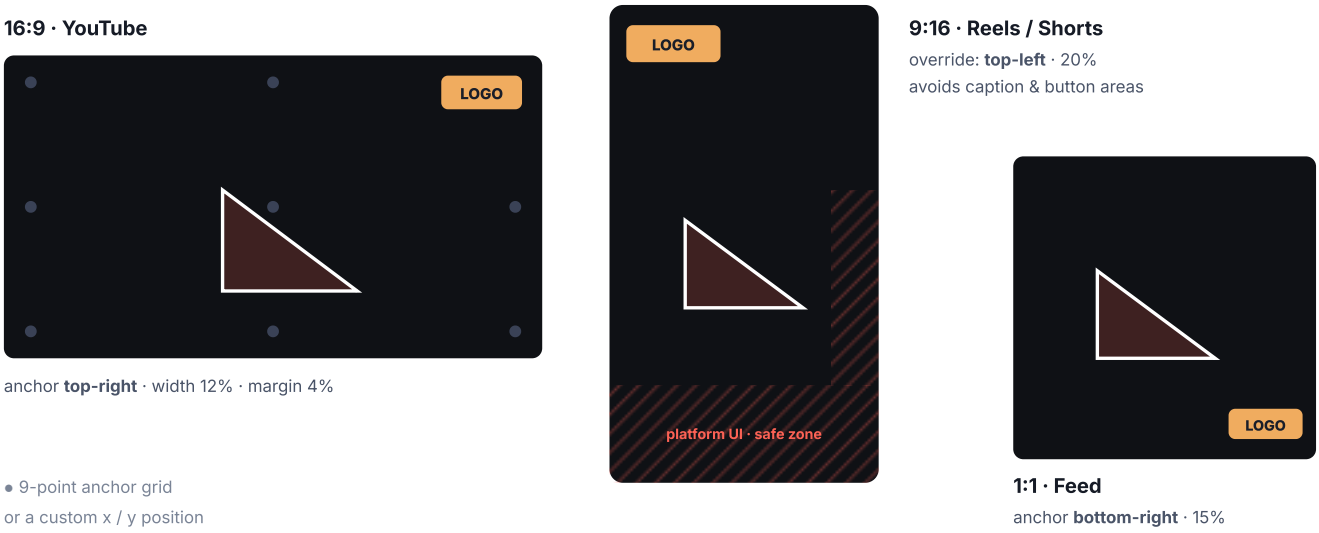
TARGET	BEST FOR	HOW YOU OPEN IT	WHERE FILES LIVE
💻 **Local PC**	Developers, decent laptops	`pnp studio` → 127.0.0.1:7860	A local folder
🐳 **Docker**	No dependency setup at all	`docker run …` → localhost	A mounted folder
☁️ **Google Colab**	Low-spec machines	Run the notebook; Studio opens via Colab's own proxy	Google Drive
🏢 **Server** later	ReelMVP production	ReelMVP web app	Postgres + S3

Why this design: zero render-server cost while we validate the product, it works for people without powerful hardware, and nothing needs rewriting when ReelMVP moves rendering onto our own servers.

04 · BRANDING

Brand kits: the logo exactly where you want it

Each brand kit holds a logo with its placement rules, a colour theme and a font. Placement can differ per format, so a reel can keep the logo clear of the app's buttons.



The same logo placed by per-format rules. The hatched area marks the platform UI zone on a 9:16 reel.

What you control

SETTING	OPTIONS
Logo file	PNG with transparency, or SVG (≤ 5 MB)
Anchor	9-point grid or custom x / y
Size, margin	% of frame width / shorter edge
Opacity	0–100%
Timing	From / until, optional fade
Per format	Different rules for 16:9, 9:16, 1:1
Theme	Background, primary, secondary, accent, text colours; font
Later	Watermark, intro/outro, safe-zone auto-nudge

What a change costs

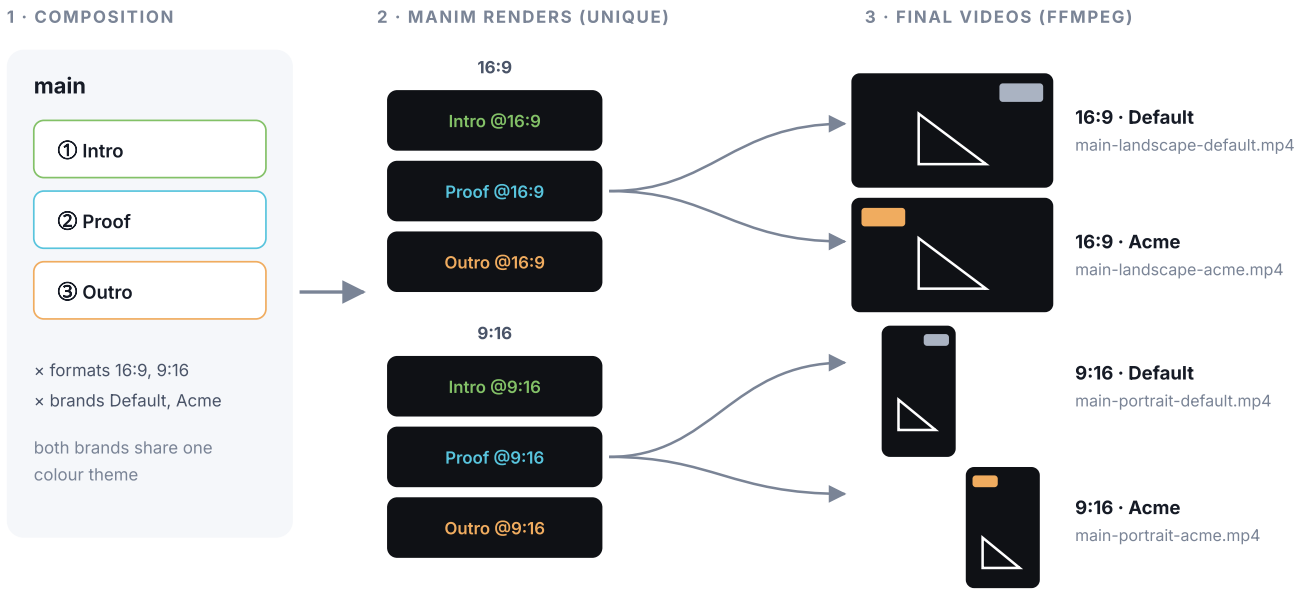
YOU CHANGE	MANIM RE-RENDER?
Logo position / size / opacity	No FFmpeg only
Clip order, music	No FFmpeg only
Brand colours / font	Yes affected scenes
Scene code, params, format	Yes affected scenes

> The logo is added at the very end, in the same FFmpeg pass that joins the scenes. Re-positioning it takes one quick encode instead of re-drawing every animation.

05 · MULTIPLE VIDEOS AND VARIATIONS

One project. Every video you need.

A project holds several **compositions** (videos), each an ordered list of scenes. An export multiplies them across formats, brands and parameters such as language. Shared work is done once.



For readability the arrows show one representative scene per format. Every final video uses all three clips of its format.

4 videos

2 formats × 2 brands of the *main* composition.

6 Manim renders, not 12

The brands share a theme, so each scene is drawn once per format and reused.

0 renders to move a logo

Only the 4 quick FFmpeg composites re-run.

How an export is described

```
# pnp.yaml (excerpt)
compositions:
  main:   { clips: [ intro.py:Intro, proof.py:Proof, outro.py:Outro ] }
  teaser: { clips: [ proof.py:Proof ], params: { title: "Why a² + b² = c²" } }

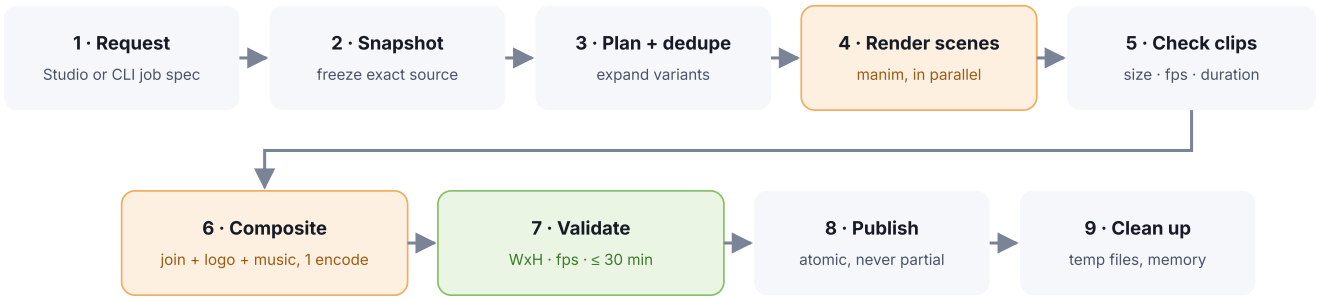
variants:
  matrix:
    composition: [main, teaser]
    format:      [landscape, portrait]     # 16:9, 9:16 (1:1 also built in)
    brand:       [default, acme]
    params: { lang: [en, hi] }             # every parameter multiplies the set
  exclude: [ { composition: teaser, format: landscape } ]
  max: 24                                  # safety cap
```

Before anything runs, the Studio shows the number of videos, the number of unique renders, and how many are already cached. You confirm, then it renders.

06 · RENDERING PIPELINE

From code to a checked MP4

Every export follows the same nine steps. A failure at any step is logged and cleaned up, and the last good video is never overwritten.



Any failure → logged with full output · temp files removed · previous good video kept · one-click retry

A scene is plain Manim

```python
from manim import *
from pnp import ctx   # format, theme, params

class Proof(Scene):
    def construct(self):
        self.camera.background_color = ctx.theme.background
        title = Text(ctx.params.get("title", "a² + b² = c²"),
                     color=ctx.theme.primary)
        # one scene, responsive to 16:9 and 9:16
        title.to_edge(UP if ctx.format.is_portrait else UL)
        self.play(Write(title))
```

Without Pi & Pixel, `ctx` falls back to defaults, so `manim -ql proof.py Proof` still works.

Safety and reliability

- **User code never runs inside the Agent.** Each render is a separate process with time, memory and process limits; cancel kills the whole tree.
- **Locked down by default.** The Agent listens only on 127.0.0.1 and every request needs a random token.
- **Secrets stay out.** Tokens and AI keys never reach the browser or the render process.
- **No fake progress bars.** Only measured counts and percentages are shown.
- **Reproducible.** Each video records its source revision and engine versions.
- **Crash-safe.** Jobs survive restarts; interrupted jobs (e.g. a Colab disconnect) can be retried.
- **Low-RAM aware.** The scheduler runs fewer jobs at once when memory is short.

07 · PLAN AND ESTIMATES

Cycle → car → truck → aeroplane

We build in stages. Each stage is usable on its own, and the next one starts only after the current stage's exit test passes.

Cycle
≈ 6 working days

- Project skeleton and tooling
- Manim + FFmpeg verified (local + Colab)
- Render one scene from the CLI
- Agent with token auth
- Studio: Render button + player
- Memory and speed benchmark

Exit: click Render, the video plays in the browser.

Car
≈ 11 working days

- Code editor, file tree, save
- Scene discovery, live logs
- Draft / preview / final
- Revisions and job queue
- Cancel, retry, crash recovery
- Join scenes, 30-min check

Exit: edit, preview, fix, then export one checked MP4.

Truck
≈ 10 working days

- Brand kits + logo upload
- Logo overlay + placement editor
- Theme passed into scenes
- 16:9 / 9:16 / 1:1 formats
- Variant matrix + dedupe
- Variant gallery

Exit: the CTO demo runs end to end.

Aeroplane
estimated after demo

- Polished Colab notebook
- Docker image + sandbox
- Security + end-to-end tests
- AI assist with Claude
- Music, intro/outro, watermark
- Server mode for ReelMVP

Exit: ready to plug into ReelMVP.



≈ 27 working days to the CTO demo; plan for 6–7 weeks with buffer. These are estimates for one developer working with Claude Code, and will be re-checked after the Cycle stage's benchmark.

CTO demo script (≈ 10 minutes, end of Truck)

#	STEP	WHAT IT PROVES
1	`pnp studio` on a laptop	No server needed; the badge shows CPU, RAM, Manim and LaTeX
2	Create "Pythagoras" starter	It is plain, editable Manim code
3	Edit, draft-preview, break, fix	Fast loop; errors link to the exact line
4	Upload logo, place it	Live placement per format, with no render
5	Export 2 formats × 2 brands	Plan preview shows 4 videos and the unique render count
6	Move the logo, export again	Zero Manim renders; only quick composites
7	Same project on Colab	Works for people without strong hardware
8	`pnp export --spec job.json`	Headless; this is how ReelMVP will call it

08 · TECHNOLOGY AND RISKS

Tech stack, risks and the road to ReelMVP

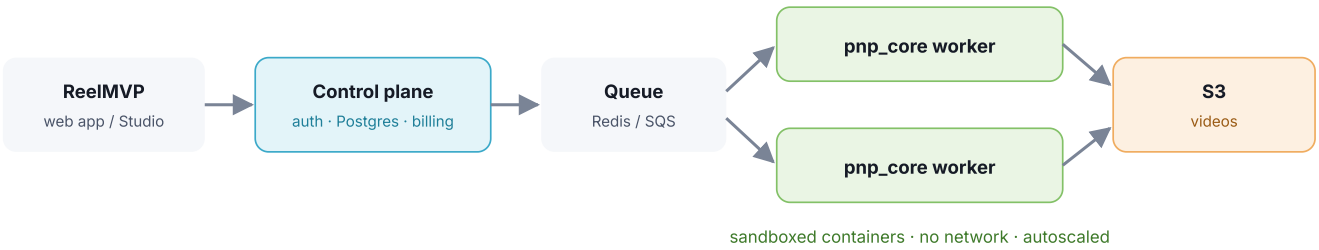
Tech stack

AREA	CHOICE
Animation	Manim Community Edition (pinned)
Media	FFmpeg 6+, ffprobe
Maths text	LaTeX (TeX Live subset / TinyTeX), optional
Backend	Python 3.12, FastAPI, Pydantic v2
Storage	SQLite + SQLAlchemy + Alembic; files / Drive
Job queue	Own SQLite queue (no Redis in MVP)
Frontend	React, TypeScript, Vite, Tailwind
Editor, terminal	Monaco, xterm.js
Quality	ruff, pyright, pytest, ESLint, Vitest, Playwright
Packaging	uv / pnp CLI, Docker, Colab notebook
AI (later)	Claude API behind a small interface

Main risks

RISK	MITIGATION
Colab's terms restrict some web UIs on the free tier	Use Colab's own proxy, check the terms in Cycle; fall back to paid Colab or local
8 GB laptops run out of memory	Draft quality first, RAM-aware scheduling, benchmarked limits
LaTeX install is slow on Colab	"Skip LaTeX" option, cached install on Drive
16:9 scenes look wrong on 9:16	`ctx.format` helpers, responsive starter scenes, gallery review
Too many variants at once	Plan preview, confirmation, cap of 24
Imported projects contain harmful code	Process limits now; Docker sandbox with no network later

Future: server mode inside ReelMVP



Unchanged: `pnp_core`, the scene SDK, the project format, the job spec and the Studio components. **Swapped:** SQLite → Postgres, local queue → Redis/SQS, disk → S3, single user → multi-tenant accounts, and sandboxing becomes mandatory.

09 · SAMPLE VIDEOS

The style we are aiming for

These videos show the visual standard Pi & Pixel targets. They are **style references, not Pi & Pixel output**: 3Blue1Brown videos are made by Grant Sanderson with ManimGL, the original library that Manim Community Edition grew out of.



Pi & Pixel demo reel: "Pythagoras in 60 seconds"
16:9 and 9:16, two brands. Rendered with Pi & Pixel itself.
Added after the 🚚 Truck stage



But what is a neural network? · 3Blue1Brown
Long-form 16:9 explainer: layered diagrams, colour-coded maths, smooth transforms.
youtube.com/watch?v=aircAruvnKk



But what is the Fourier Transform? A visual introduction · 3Blue1Brown
Graphs, axes and animated functions. The core of what Manim does well.
youtube.com/watch?v=spUNpyF58BY



But what is a Fourier series? From heat flow to drawing with circles · 3Blue1Brown
Complex motion built from simple parts. A good benchmark for long, heavy scenes.
youtube.com/watch?v=r6sGWTCMz2k



Essence of linear algebra (playlist) · 3Blue1Brown
A multi-video series. The kind of course ReelMVP could produce as one project.
youtube.com/playlist?list=PLZHQObOWTQDPD3MizzM2xVFitgF8hE_ab



Manim Community example gallery
Short clips with their full source code: exactly the engine Pi & Pixel runs.
docs.manim.community/en/stable/examples.html

Project links

Repository	github.com/harshdotcom/PiAndPixel
Architecture	`docs/architecture.md`
Requirements	`PRD.txt` + `docs/requirements.md`
Build plan	`AGENTS.md` §4
Manim CE	github.com/ManimCommunity/manim

Glossary

Agent	Python program that renders, on your PC or Colab
Studio	The browser app that controls the Agent
Composition	One video: an ordered list of scenes
Variant	One output: video × format × brand × params
Render unit	One scene drawn once by Manim, reused